OPERATING SYSTEMS

Project Year 2025–2026

The Citadel System

Phase 1: Valar Compilis

Design & Implementation Report

Author :

Anas TAGUI | Abderrahmen MESKINE

La Salle Universitat Ramon Llull

March 2026

Table of Contents

1. Introduction

2. System Architecture Overview

3. Data Structures and Justification

3.1 MaesterConfig

3.2 Inventory and Product

3.3 TradeList and TradeItem

4. Module Design

4.1 globals.h: Shared Definitions

4.2 io_utils: I/O Utilities

4.3 config: Configuration Parser

4.4 inventory: Stock Management

4.5 commands: Command Processor

4.6 main: Entry Point

5. Command Terminal

5.1 Command Recognition Logic

5.2 START TRADE Sub-Menu

6. File Formats

6.1 maester.dat (Configuration)

6.2 stock.db (Binary Inventory)

7. Signal Handling and Graceful Shutdown

8. Compilation and Testing

9. Observed Problems and Solutions

10. Conclusions

1. Introduction

The Citadel System is a distributed network simulation inspired by the world of Westeros. The project models a decentralized communication system where independent realms (Great Houses) can forge alliances and conduct trade through a network of Maester processes.

Phase 1, titled Valar Compilis, focuses on building the local foundations of the Maester process. At this stage, no network communication is implemented. The objective is to correctly read configuration and inventory files, store data in well-designed structures, and provide an interactive command terminal that recognizes and processes all specified commands.
The program is written entirely in C, using only low-level system calls (read() and write()) for all I/O operations.

2. System Architecture Overview

The Phase 1 architecture is a single-process application composed of six modular components. Each module is separated into a .c implementation file and a .h header file.

Module	Files	Responsibility
globals	globals.h	Shared type definitions and constants
io_utils	io_utils.c / .h	Read/write wrappers, string utilities
config	config.c / .h	Parsing maester.dat configuration
inventory	inventory.c / .h	Reading/displaying binary stock.db
commands	commands.c / .h	Command parsing and execution
main	main.c / .h	Entry point, signal handling, main loop

`The dependency flow is: main → commands → config / inventory → io_utils → globals. Each`

module only includes headers it directly depends on, and globals.h is the single point where all system headers are included.

The program execution flow is: (1) validate command-line arguments, (2) install signal handler, (3) parse maester.dat, (4) load stock.db, (5) enter the interactive command loop, (6) clean up and exit.

3. Data Structures and Justification

3.1 MaesterConfig

The MaesterConfig structure stores all information parsed from the maester.dat configuration file. It contains the realm name, folder path, number of envoys, the Maester's own IP and port, and a fixed-size array of Route structs representing the routing table.

```
// Custom Type Definitions
typedef struct {
    char sRealmName[MAX_REALM_NAME];
    char sIp[MAX_IP];
    int nPort;
} Route;

typedef struct {
    char sRealmName[MAX_REALM_NAME];
    char sFolderPath[MAX_PATH];
    int nNumEnvoys;
    char sIp[MAX_IP];
    int nPort;
    Route aRoutes[MAX_ROUTES];
    int nNumRoutes;
    int nDefaultRouteIndex;
} MaesterConfig;
```

Justification: A fixed-size array of routes (max 50) is used because the number of realms in the network is expected to be small and known at startup. This avoids unnecessary dynamic allocation complexity while keeping the routing table easily iterable. The DEFAULT route index is stored separately for O(1) access during message forwarding in later phases.

3.2 Inventory and Product

The Product structure mirrors the binary record format of stock.db: a 100-byte character name, an integer amount, and a float weight. The Inventory wraps a **dynamically allocated array** of Products.

```
typedef struct {
    char sName[MAX_PRODUCT_NAME];
    int nAmount;
    float fWeight;
} Product;

typedef struct {
    Product *pProducts;
    int nNumProducts;
} Inventory;
```

Justification: A dynamic array (using malloc/realloc) is used because the number of products in the stock file is not known at compile time. The array starts with capacity 16 and doubles when needed. This provides contiguous memory layout for cache-friendly sequential access during display operations, which is the primary access pattern.

3.3 TradeList and TradeItem

The TradeList is a temporary structure used during the START TRADE sub-menu to accumulate the user's shopping list before writing it to a file.

```
typedef struct {
    char sProductName[MAX_PRODUCT_NAME];
    int nAmount;
} TradeItem;

typedef struct {
    TradeItem aItems[MAX_TRADE_ITEMS];
    int nNumItems;
    char sTargetRealm[MAX_REALM_NAME];
} TradeList;
```

Justification: A fixed-size array of 50 items is sufficient for a trade session. The structure is stack-allocated and lives only during the trade sub-menu execution, so no dynamic memory management is needed.

4. Module Design

4.1 globals.h: Shared Definitions

This header-only module centralizes all system includes (unistd.h, stdlib.h, string.h, fcntl.h, signal.h, ctype.h), constant definitions (buffer sizes, max values), and all typedef struct declarations. Other modules include only globals.h instead of individual system headers, following the style guide's recommendation.

4.2 io_utils: I/O Utilities

Since `printf/scanf` are forbidden, this module provides replacement functions built on `read()` and `write()`:

Function	Purpose
`printMessage()`	Writes a string to stdout via write()
`printInt()`	Converts int to string, writes to stdout
`printFloat()`	Converts float to string (1 decimal), writes to stdout
`readLine()`	Reads stdin/file char-by-char until \n or EOF, returns malloc'd string
`stringToInt()`	Converts a string to integer (replaces atoi)
`compareIgnoreCase()`	Case-insensitive string comparison using tolower()
`trimWhitespace()`	Removes leading/trailing spaces and tabs
`countWords()`	Counts whitespace-separated words in a string
`getWord()`	Extracts the nth word from a string (0-indexed)
`stripAmpersand()`	Removes all '&' chars from realm names (protocol requirement)

4.3 config: Configuration Parser

The parseConfig() function opens the maester.dat file and reads it line-by-line using readLine(). The first five lines have a fixed meaning (realm name, folder path, envoy count, IP, port), followed by the --- ROUTES --- separator and then route entries. Each route line is parsed into 3 words: realm name (or DEFAULT), IP, and port. The '&' character is stripped from all realm names at parse time to prevent protocol conflicts in later phases.

4.4 inventory: Stock Management

The loadInventory() function reads the binary stock.db file record by record. Each record is exactly 108 bytes: 100 bytes for the name (char[100]), 4 bytes for quantity (int), and 4 bytes for weight (float). Records are read using read() and stored in a dynamically growing array. The displayInventory() function formats the output as a table matching the project specification's Trade Ledger format.

4.5 commands: Command Processor

The processCommand() function is the central dispatcher. It trims the input, counts words, extracts the first two words, and uses a series of case-insensitive comparisons to identify the command. Importantly, it

checks the **exact word count** for each command to reject inputs with extra arguments (e.g., "LIST REALMS please" → Unknown command). Commands fully implemented in Phase 1 are: LIST REALMS, LIST PRODUCTS (own), START TRADE, and EXIT. Others print "Command OK".

4.6 main: Entry Point

The main() function validates the 2 required arguments (config file and stock file), installs the SIGINT handler using sigaction(), calls the parser and loader, prints the initialization message, and enters the command loop. On exit (either via the EXIT command or CTRL+C), all dynamically allocated memory is freed.

5. Command Terminal

5.1 Command Recognition Logic

All commands are case-insensitive. The recognition follows a strict word-count matching approach to distinguish valid commands from unknown input:

Command	Words	Phase 1 Behavior
LIST REALMS	2	Lists all realms from the routing table
LIST PRODUCTS	2	Displays own inventory (Trade Ledger)
LIST PRODUCTS <realm>	3	Prints "Command OK" (stub)
PLEDGE <realm> <sigil>	3	Prints "Command OK" (stub)
PLEDGE RESPOND <realm> ACCEPT/REJECT	4	Prints "Command OK" (stub)
PLEDGE STATUS	2	Prints "Command OK" (stub)
ENVOY STATUS	2	Prints "Command OK" (stub)
START TRADE <realm>	3	Enters trade sub-menu
START TRADE (no args)	2	Error: missing arguments
EXIT	1	Graceful shutdown with farewell message

Any input with extra words beyond what's expected (e.g., "START TRADE TheVale now" with 4 words instead of 3) is rejected as "Unknown command". Any completely unrecognized command (e.g., "CHECK MY REALMS") also returns "Unknown command".

5.2 START TRADE Sub-Menu

When START TRADE is invoked with a valid realm name, the prompt changes from $ to `(trade)>` . The sub-menu supports four commands:

Sub-Command	Description
`add <product> <amount>`	Adds a product to the shopping list (multi-word names supported)
`remove <product> <amount>`	Reduces or removes a product from the list
`send`	Writes the trade list to a text file and exits sub-menu
`cancel`	Discards the list and exits sub-menu

The send command writes a plain text file named `trade_<RealmName>.txt` in the Maester's working directory (specified in maester.dat). Each line contains `ProductName Amount`. This file will later be transmitted over the network in Phase 3.

6. File Formats

6.1 maester.dat (Configuration File)

A plain text file with a fixed structure. The parser reads it line by line:

Line	Content	Example
1	Realm name	Dragonstone
2	Working directory path	/dragonmont
3	Number of envoys	3
4	Listening IP address	192.168.1.3
5	Listening port	9003
6	Separator	--- ROUTES ---
7+	Route: Name IP Port	KingsLanding 192.168.1.5 9002
	or Default route	DEFAULT 192.168.1.5 9002
	or Unknown route	Driftmark *.*.*.* 0

6.2 stock.db (Binary Inventory File)

A binary file where each record is exactly **108 bytes**, laid out sequentially with no separators:

Field	Type	Size	Description
Name	char[100]	100 bytes	Product name, null-padded
Amount	int	4 bytes	Quantity (gold value)
Weight	float	4 bytes	Weight in stone

The file is read using read() in chunks matching each field's size. The total number of products is determined by reading until EOF.

7. Signal Handling and Graceful Shutdown

The Maester process can be terminated in two ways: via the EXIT command or by pressing CTRL+C (which sends SIGINT). In both cases, the program must release all resources and exit cleanly.

We use sigaction() (rather than the less reliable signal()) to install a SIGINT handler at startup. The handler prints a farewell message, calls freeInventory() and freeConfig() to release all dynamic memory, and then calls _exit(0) to terminate immediately. The global config and inventory structures are accessible to the handler through file-scope static variables.
The EXIT command follows the same cleanup path but through the normal return flow: it sets the gnRunning flag to 0, which breaks the main loop, and the cleanup code after the loop frees all resources before `main()` returns 0.

8. Compilation and Testing

First we need the Gx_F1.tar File, the configuration file and the inventory database

```
anas.tagui@matagalls:~/carpetaDestino/phase1>ls
Gx_F1.tar  maester.dat  stock.db
anas.tagui@matagalls:~/carpetaDestino/phase1>
```

then we need to extract the Gx_F1.tar file so we can see the other files

```
anas.tagui@matagalls:~/carpetaDestino/phase1>tar xf Gx_F1.tar
anas.tagui@matagalls:~/carpetaDestino/phase1>ls
commands.c  config.c  globals.h  inventory.c  io_utils.c  maester.dat  main.h    stock.db
commands.h  config.h  Gx_F1.tar  inventory.h  io_utils.h  main.c       Makefile
anas.tagui@matagalls:~/carpetaDestino/phase1>
```

The project compiles with gcc -Wall -Wextra -Werror -pedantic and produces **zero warnings**. The Makefile tracks all header dependencies to ensure correct incremental builds.

Compilation commands:

```
anas.tagui@matagalls:~/carpetaDestino/phase1>make clean
rm -f main.o io_utils.o config.o inventory.o commands.o Maester
anas.tagui@matagalls:~/carpetaDestino/phase1>make
gcc -Wall -Wextra -Werror -pedantic -g -c main.c
gcc -Wall -Wextra -Werror -pedantic -g -c io_utils.c
gcc -Wall -Wextra -Werror -pedantic -g -c config.c
gcc -Wall -Wextra -Werror -pedantic -g -c inventory.c
gcc -Wall -Wextra -Werror -pedantic -g -c commands.c
gcc -Wall -Wextra -Werror -pedantic -g -o Maester main.o io_utils.o config.o inventory.o commands.o
anas.tagui@matagalls:~/carpetaDestino/phase1>./Maester maester.dat stock.db
Maester of 3 initialized. The board is set.
$
```

All test cases from the Phase 1 testing document pass correctly:

```
Maester of 3 initialized. The board is set.
$ LIST REALMS
- TheVale
- Driftmark
$ PLEDGE TheVale arryn.png
Command OK
$ PLEDGE RESPOND TheVale ACCEPT
Command OK
$ PLEDGE RESPOND TheVale REJECT
Command OK
$ LIST PRODUCTS KingsLanding
Command OK
$ LIST PRODUCTS
--- Trade Ledger ---
Item                          | Value (Gold) | Weight (Stone)
-------------------------------------------------------
Antlered Helm                 | 250          | 4.5
Warhammer of Bronze           | 400          | 8.0
Stormlands Ale Barrel         | 80           | 10.0
Ship Timber Plank             | 60           | 15.0
Salted Fish Barrel            | 55           | 20.0
Wine Cask                     | 120          | 5.0
Leather Riding Boots          | 90           | 2.0
Bronze Helm                   | 140          | 3.5
Woolen Blanket                | 30           | 2.5
Iron Dagger                   | 35           | 1.2
Grain Sack                    | 40           | 10.0
-------------------------------------------------------
Total Entries: 11
$ |
```

```
$ START TRADE Driftmark
(trade)> |
```

```
anas.tagui@matagalls:~/carpetaDestino/phase1>./Maester maester.dat stock.db
Maester of 3 initialized. The board is set.
$ PLEDGE STATUS
Command OK
$ ENVOY STATUS
Command OK
$ EXIT
The Maester of 3 signs off. The ravens rest.
anas.tagui@matagalls:~/carpetaDestino/phase1>
```

```
anas.tagui@matagalls:~/carpetaDestino/phase1>./Maester maester.dat stock.db
Maester of 3 initialized. The board is set.
$ LiSt REAlms
- TheVale
- Driftmark
$ PLedgE TheVale arryn.png
Command OK
$ START TRADE
Missing arguments, can't start a trade. Please review the syntax.
$ START TRADE (no args)
Unknown command
$ CHECK MY REALMS
Unknown command
$ START TRADE TheVale now
Unknown command
$ LIST REALMS please
Unknown command
$ |
```

Test Input	Expected Output	Result
LIST REALMS	- TheVale / - KingsLanding / - Driftmark	PASS
PLEDGE TheVale arryn.png	Command OK	PASS
PLEDGE RESPOND TheVale ACCEPT	Command OK	PASS
PLEDGE RESPOND TheVale REJECT	Command OK	PASS
LIST PRODUCTS KingsLanding	Command OK	PASS
LIST PRODUCTS	Prints inventory table (11 entries)	PASS
START TRADE Driftmark	Enters (trade)> sub-menu	PASS
PLEDGE STATUS	Command OK	PASS
ENVOY STATUS	Command OK	PASS
EXIT	Farewell message, clean exit	PASS
LiSt REAlms	Same as LIST REALMS (case-insensitive)	PASS
PLedgE TheVale arryn.png	Command OK (extra spaces handled)	PASS
START TRADE (no args)	Missing arguments error message	PASS
CHECK MY REALMS	Unknown command	PASS
START TRADE TheVale now	Unknown command	PASS
LIST REALMS please	Unknown command	PASS

9. Observed Problems and Solutions

Problem: No printf/scanf available **Solution:** Built a complete I/O utility library (io_utils module) with printMessage(), printInt(), printFloat(),
and readLine() functions that exclusively use read() and write() system calls. Integer-to-string conversion is done manually using modulo arithmetic and digit reversal.

Problem: Multi-word product names in trade commands

Solution: The add and remove commands in the trade sub-menu need to handle product names like "Myrish Lace" where the name consists of multiple words. The solution: the last word is always the amount, and all words between the command and the last word form the product name.

Problem: Case-insensitive command matching with strict argument counts

Solution: Used a custom compareIgnoreCase() function that compares character-by-character using tolower(). Combined this with word counting to enforce exact argument counts — preventing commands with extra trailing words from being accepted.

Problem: Extra whitespace between arguments (e.g., 'PLedgE TheVale')

Solution: The word extraction functions (countWords, getWord) treat any sequence of spaces/tabs as a single delimiter, so multiple consecutive spaces are naturally handled without special logic.

Problem: Memory management and leak prevention

Solution: Every readLine() call returns a malloc'd string that must be freed after use. The main loop consistently frees each line after processCommand() returns. The signal handler also frees the inventory's dynamic array before exiting.

10. Conclusions

Phase 1 of The Citadel System has been successfully implemented with all required functionality. The Maester process correctly reads both configuration and inventory files, stores data in appropriate structures, and provides a robust interactive terminal that handles all specified commands with proper case-insensitivity and error handling.

The modular architecture has been designed with future phases in mind. The routing table in MaesterConfig will be used for socket-based message forwarding in Phase 2. The inventory system will support stock updates when trade processing is implemented in Phase 3. The envoy count stored in the config will drive the process creation in Phase 4.
The code compiles cleanly with all warnings enabled, follows the La Salle style guide (Hungarian notation, file/function headers, proper indentation), and passes all test cases from the Phase 1 testing document. The system handles edge cases such as empty input, extra whitespace, unknown commands, and incomplete commands with appropriate error messages.